

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
Государственное бюджетное профессиональное
образовательное учреждение города Москвы
«МОСКОВСКИЙ КОЛЛЕДЖ БИЗНЕС-ТЕХНОЛОГИЙ»
(ГБПОУ КБТ)

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме: Внедрение системы автоматизации развертывания, масштабирования и управления контейнеризованными приложениями в ЗАО «Эвалар»

СПЕЦИАЛЬНОСТЬ: 09.02.06 «Сетевое и системное администрирование»

ГРУППА: СА41-19

ИСПОЛНИТЕЛЬ:

Чучканов Т. С.

РУКОВОДИТЕЛЬ:

Подвопросом Н. Н.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ, МЕТОДОЛОГИИ И ТЕХНОЛОГИИ.....	4
1.1 Методология DevOps и ее роль в современном мире.....	4
1.2 Контейнеризация и виртуализация. Сферы применения и основные технологии.....	8
1.3 Автоматизация управления конфигурацией. Понятие инфраструктуры как кода.....	12
ГЛАВА 2. МОДЕРНИЗАЦИЯ ИНФРАСТРУКТУРЫ КОМПАНИИ ЗАО «ЭВАЛАР». СОЗДАНИЕ КУЛЬТУРЫ DEVOPS.....	15
2.1 Анализ имеющейся инфраструктуры и подхода к работе. Выделение проблем.....	15
2.2 Поиск способов решения выделенных проблем. Разработка плана модернизации.....	19
2.3 Начало реализации плана по модернизации инфраструктуры.....	28
2.3.1 Выбор проекта.....	28
2.3.2 Проектирование кластера.....	29
2.3.3 Развертывание кластера.....	33
2.3.4 Портинг выбранного проекта.....	35
2.3.5 Перенос стендов разработки.....	38
2.4 Дальнейшее развитие проекта.....	41
ГЛАВА 3. ЭКОНОМИЧЕСКАЯ ЧАСТЬ.....	43
ГЛАВА 4. ОХРАНА ТРУДА И ТЕХНИКА БЕЗОПАСНОСТИ.....	44
ЗАКЛЮЧЕНИЕ.....	45
БИБЛИОГРАФИЧЕСКИЙ СПИСОК.....	46

ВВЕДЕНИЕ

ГЛАВА 1. ОСНОВНЫЕ ПОНЯТИЯ, МЕТОДОЛОГИИ И ТЕХНОЛОГИИ

1.1 Методология DevOps и ее роль в современном мире

Для лучшего понимания дальнейшего материала, представленного в данной выпускной квалификационной работе, для начала раскроем ряд используемых понятий.

DevOps — (от англ. Development и Operations) это набор практик и инструментов, подход к работе, который позволяет компании наладить эффективное взаимодействие сотрудников, занимающихся разработкой ПО, с другими IT-специалистами. Это позволяет производить создание и улучшение продуктов быстрее, чем это происходит, используя более старые и распространенные подходы к разработке программных продуктов.[1] [2]

Если рассматривать с точки зрения инструментов, то суть этой методологии заключается в автоматизации и интегрировании рабочих процессов между командами разработки и другими командами IT.

Относительно же взаимодействия между командами, они более не являются «изолированными». То есть члены различных команд отдела IT, а именно, разработки и администрирования, более тесно взаимодействуют друг с другом, как если бы они были членами одной команды. Более того, в некоторых случаях эти команды действительно совмещают в одну команду, где каждый имеет навыки из нескольких дисциплин.

Делается это для исключения проблем, вызываемых разрозненной работой команд, приводящих к замедлению работы в целом, а также для наоборот ускорения, так как процессы работы над разрабатываемыми продуктами и их поставки конечному пользователю являются непрерывными благодаря различным средствам автоматизации процессов.

Подобный подход может включать и взаимодействие с другими командами. Так, например, при тесной интеграции с командой безопасности,

получается DevSecOps, где информационная безопасность занимает центральную роль, и все процессы организуются с расчетом на защищенность.

Тем не менее, инструменты не являются основным элементом DevOps. Самая важная основополагающая DevOps — это именно подход к взаимодействию сотрудников, культура. Столь сильная согласованность и слаженность работы людей разных направлений требует мотивации и стараний со всех сторон для достижения максимального эффекта, ведь зачастую для такого необходимо пересмотреть уже устоявшиеся привычные вещи.

С ростом популярности и востребованности такой методологии начали появляться и новые специалисты, такие как DevOps-инженеры, занимающиеся налаживанием непрерывности процессов между разработчиками и администраторами.

Так называемый «жизненный цикл DevOps» можно описать как продолжительное циклическое выполнение действий восьми непрерывных этапов*:

- 1) Планирование (Plan);
- 2) Создание/разработка (Code/Create);
- 3) Сборка (Build);
- 4) Тестирование (Test/Validate);
- 5) Выпуск/публикация (Release);
- 6) Развертывание (Deploy);
- 7) Эксплуатация (Operate);
- 8) Мониторинг (Monitor).

Для удобства восприятия данный цикл зачастую изображается в виде всех этапов, расположенных на символе бесконечности (∞). На Рисунке 1 представлен один из примеров подобных схем.

*Это один из множества вариаций описания цикла DevOps. Могут быть незначительные модификации.



Рисунок 1 — Пример схемы, визуализирующей этапы жизненного цикла DevOps, найденной в глобальной сети Интернет

В ходе выполнения работы был осуществлен анализ отчета «State of DevOps Report»[3] компании Puppet Labs Limited за 2021, публикуемый раз в несколько лет на основе производимых исследований (см. Рисунок 2).

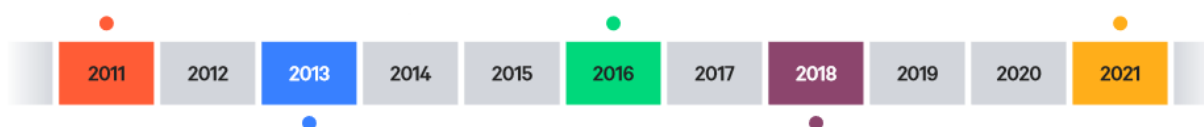


Рисунок 2 — Годы публикации отчетов исследования

Согласно этому отчету, внедрение DevOps лучше всего происходит по модели снизу вверх, начиная с самих команд, и распространяясь на всю организацию. Без поддержки со стороны верхних руководящих лиц, компании не получится сильно эволюционировать относительно применяемых подходов к осуществлению рабочей деятельности.

Также компании, внедряющие DevOps, можно условно разделить на три категории по уровню эволюции: низкий, средний, высокий. На разных этапах такой эволюции выделяются схожие факторы, препятствующие дальнейшему процессу модернизации. Среди них можно выделить:

- Неприязнь изменений со стороны коллектива/руководства;
- Старая архитектура;
- Нехватка навыков;

- Ограниченная автоматизация;
- Неясные цели.

Причем, чем выше компания по уровню эволюции принятия DevOps, тем меньше барьеров связаны с культурой внутри компании, и с неясностью целей. Основными барьерами остаются только старая архитектура и нехватка навыков, которые со временем тоже устраняются.

Многие компании останавливаются на среднем уровне внедрения DevOps.

Более подробное рассмотрение данного отчета выходит за рамки этой работы. Поскольку он находится в свободном доступе, ознакомиться с ним может любой желающий в любое время.

1.2 Контейнеризация и виртуализация. Сферы применения и основные технологии

Разберем понятия виртуализации компьютерных систем (виртуальные машины) и контейнеризации, их различия и базовые принципы работы, почему оба подхода к виртуализации популярны. После этого выделим, за счет чего контейнеризация вытеснила виртуальные машины в некоторых сферах, и почему это хорошо для индустрии в целом.

Виртуализация — это процесс более эффективного использования вычислительных ресурсов компьютерной системы за счет разделения этих ресурсов путем применения дополнительных уровней абстракции.

Виртуальные машины — виртуальное представление, или эмуляция, физических компьютерных систем со своими виртуальными компонентами, такими как центральный процессор или оперативная память. Они не имеют возможности напрямую взаимодействовать с физическим компьютером, на котором развернуты. Вместо этого взаимодействие происходит при помощи гипервизора.[4]

Гипервизор — технология, которая служит уровнем абстракции между виртуальными машинами (гостями) и нижележащей компьютерной системой (хостом). Гипервизоры выделяют ресурсы компьютерной системы гостевым системам по необходимости.

Существует два вида гипервизоров: первого типа и второго.

Гипервизоры первого типа — легковесные операционные системы, имеющие лишь компоненты, необходимые для обеспечения виртуализации. Они не могут быть использованы как обычные серверные или настольные операционные системы.

Типичные примеры:

- Linux с модулем KVM (например, Proxmox VE, использующий qemu);
- VMWare ESXi;

- Xen;
- Hyper-V.

Гипервизоры второго типа — специальные программы, устанавливаемые поверх существующей операционной системы. Их можно установить на обычные рабочие станции, не влияя на возможность выполнения ежедневных задач.

Типичные примеры:

- VMWare Workstation;
- VirtualBox.

За счет отсутствия лишних компонентов, и некоторых других аспектов гипервизоры первого типа более производительны, и зачастую гостевые системы работают с производительностью очень близко сопоставимой с той, которую можно было бы достичь при использовании системы не как гостевой, а на компьютерной системе непосредственно. Благодаря этому такие гипервизоры используются для развертывания множества виртуализированных серверов. Это позволяет компаниям реализовывать более гибкую и защищенную инфраструктуру, и использовать максимум имеющихся вычислительных ресурсов с минимальным простоем.

Контейнеризация — один из видов легковесной виртуализации на уровне операционной системы или приложения, который работает путем изоляции одного или нескольких процессов.[5] В данном контексте слово «контейнеризация» будет использоваться для описания контейнеризации операционных систем, но не приложений, и относительно операционных систем на базе ядра Linux. Стоит упомянуть, что в других системах имеются схожие технологии. Так, в ОС FreeBSD имеются «Jails», что в дословном переводе означает «тюрьмы».

Отличие контейнеров от виртуальных машин в том, что они используют одно ядро — операционной системы, на которой запущены. Невозможно запустить контейнер с системой процессорной архитектуры, отличной от

архитектуры системы хоста, на котором разворачивается контейнер. Это же применимо и к самой операционной системе. Это означает, что на компьютере с установленным дистрибутивом Fedora для процессорной архитектуры x86_64 возможно запустить контейнер с Debian или любым другим дистрибутивом для этой архитектуры. Запустить же на такой системе контейнер FreeBSD или для любой другой архитектуры, такой как ARM64, невозможно.

Контейнеры обычно создаются из неизменяемых образов, имеющих файловые системы только для чтения, и содержат только приложение (или несколько), которые планируется разворачивать с их помощью, а также только самые необходимые программы и библиотеки, нужные для работы этого приложения.

Для создания и запуска контейнеров используются низкоуровневые среды выполнения — «рантаймы» (от англ. runtime), используемые более высокоуровневыми средами выполнения, которые добавляют функционал для работы с образами контейнеров, сетевые возможности, и многое другое. Для простого обращения с такими средами выполнения разрабатываются специализированные инструменты, позволяющие разработчикам и системным администраторам работать с контейнерами, не углубляясь в технические сложности.[6]

Существует множество различных сред выполнения контейнеров, рассчитанных на разные сценарии применения, и имеющие разную степень защиты. Одни из самых распространенных низкоуровневых сред выполнения контейнеров — runc и LXC, а среди более высокоуровневых — containerd, CRI-O и LXD. Самыми распространенными высокоуровневыми инструментами для работы с контейнерами являются Docker и Podman.

Тем не менее, вне зависимости от конкретной реализации сред выполнения контейнеров, все они базируются на двух специфических принципах — namespaces и cgroups.

Namespaces (пространства имен) — это абстракции над глобальными ресурсами операционной системы, разделяющие и изолирующие их. Доступ к ресурсам внутри изолированных пространств имен есть у процессов-участников этих пространств, но не у других процессов. Это позволяет процессам иметь, например, собственные изолированные сетевые интерфейсы, такие как `loopback`, либо идентификаторы процессов.[7]

`cgroups` (от англ. control groups, контрольные группы) — механизм ядра Linux для иерархической организации процессов и контролируемого, конфигурируемого распределения системных ресурсов в этой иерархии. Контрольные группы образуют древовидную структуру, и каждый процесс в системе одновременно может принадлежать только к одной контрольной группе. Все потоки и дочерние процессы принадлежат той же контрольной группе, что и родительский процесс. При этом, возможно выполнение миграции процесса в другую контрольную группу.[8]

Суммарно, пространства имен — механизм, позволяющий ограничить видимость ресурсов, а контрольные группы ограничивают, какие ресурсы и в каком количестве могут быть использованы.

В современном мире, среди множества других способов применения, контейнеры также используются для: развертывания сред разработки (Development containers — контейнеры разработки), временных сред сборки ПО, сред для работы конечных программных продуктов, и так далее. Поскольку контейнеризованный подход к развертыванию программного обеспечения позволяет быстро и удобно разворачивать новые версии ПО, и удалять старые, собирать и распространять образы, компании и отдельные люди все чаще склоняются к использованию контейнеризованного серверного ПО. Где раньше потенциально потребовалось бы каждый раз вручную устанавливать и конфигурировать приложения, теперь можно за несколько секунд получить такой же результат от пары команд по развертыванию контейнера. Также развитие этой технологии открыло и новые возможности, дало толчок к

появлению новых технологий, использующих контейнеры как один из главных компонентов.

В некоторых областях виртуальные машины были вытеснены контейнерами. Но полное вытеснение маловероятно, так как эти два подхода к виртуализации нацелены на разные задачи, и скорее дополняют друг друга, чем конкурируют.

1.3 Автоматизация управления конфигурацией. Понятие инфраструктуры как кода

При традиционном подходе, IT-команды полагаются на ручное развертывание инфраструктуры и ручную конфигурацию этой инфраструктуры, требующие совершения большого количества монотонных однообразных действий. Такой подход является неэффективным и легко подверженным ошибкам, зачастую результирующим в увеличенном простое и, соответственно, нарушении работы бизнеса, что в свою очередь может привести к потерям в денежном эквиваленте.

В современном мире, с каждым годом сложность применяемых в компаниях систем и нагрузка на них постоянно растут, а вместе с ними и количество конфигураций, которые необходимо реализовывать, наряду с увеличением инфраструктуры. Требования к масштабируемости инфраструктуры и эффективности работы с ней также продолжают расти.

Когда администраторам приходится обслуживать большое количество компьютерных систем и сервисов, работающих на них, зачастую превосходящее по размерам само количество обслуживающего персонала, и часто вносить изменения или осуществлять действия по развертыванию новой инфраструктуры, вероятность возникновения ошибок повышается. Эффективность работы же, напротив, уменьшается. Это требует создания новых инновационных подходов к автоматизации.

Постепенно создаются новые методы решения таких проблем. Одними из самых влиятельных являются IaC (от англ. Infrastructure as Code, рус. — инфраструктура как код) и Configuration Management (управление конфигурациями).

IaC — это современный подход к управлению IT-инфраструктурой, при котором инфраструктура описывается при помощи кода.

В отличие от привычного подхода, при котором все изменения и настройки осуществляются администраторами вручную и распределены по разным объектам инфраструктуры, IaC позволяет описывать ее состояние в более унифицированном и контролируемом виде, а затем автоматизированно выполнять развертывание и конфигурацию для достижения этого состояния, используя системы развертывания и системы управления конфигурацией.[9] [10] Как правило, конфигурационные файлы и файлы с описанием требуемого состояния хранятся в специально отведенных репозиториях, или другими словами хранилищах кода, систем управления версиями, что позволяет иметь единый источник достоверной конфигурации, и отслеживать историю изменений, а также осуществлять восстановление к предыдущим версиям по необходимости.

Благодаря этому, даже самые сложные инфраструктуры могут быть созданы и сконфигурированы быстро и незатруднительно.

В то время как одни инструменты специализируются на автоматизации развертывания инфраструктуры, другие разработаны для ее конфигурации, но встречаются решения, которые совмещают в себе оба функционала. Так, Terraform и Pulumi — типичные представители инструментов по развертыванию инфраструктуры. Ansible, Puppet, Chef — системы управления конфигурацией.

Использование подобных инструментов направлено на:

- повышение скорости работы обслуживающего персонала при развертывании и настройке компонентов инфраструктуры;

- уменьшение объемов обслуживаемого персонала, требуемого для работы с инфраструктурой.
- снижение вероятности возникновения возможных ошибок, вызванных человеческим фактором;
- обеспечение возможности применения практик разработки ПО к управлению конфигурацией инфраструктуры;
- значительное упрощение развертывания и настройки сложных систем;
- возможность версионирования инфраструктуры и отслеживания изменений путем применения систем управления версиями;
- избежание неоднородности подхода к конфигурации;
- поддержание заданного состояния инфраструктуры;
- повторное использование элементов конфигурации на других компонентах инфраструктуры;
- а также многое другое.

ГЛАВА 2. МОДЕРНИЗАЦИЯ ИНФРАСТРУКТУРЫ КОМПАНИИ ЗАО «ЭВАЛАР». СОЗДАНИЕ КУЛЬТУРЫ DEVOPS

2.1 Анализ имеющейся инфраструктуры и подхода к работе. Выделение проблем

После ознакомления с базовыми терминами и их определениями, а также видами технологий, с которыми предстоит встретиться во время работы с инфраструктурой, можно приступить непосредственно к рассмотрению текущего состояния инфраструктуры компании ЗАО «Эвалар» и подхода членов команд ИТ-отдела к поддержке имеющейся инфраструктуры и развертыванию новой.

В ходе прохождения преддипломной производственной практики был осуществлен сбор информации об имеющейся инфраструктуре и подходах к работе с ней. Получение информации осуществлялось как путем наблюдения за процессом работы некоторых сотрудников, так и при живом опросе некоторых членов команды, в ходе которого была проведена дискуссия со следующими сотрудниками:

- Руководителем группы разработки Битрикс;
- Заместителем директора по ИТ-инфраструктуре;
- Инженером по поддержке и разработке ПО.

После проведения анализа полученной информации, собранной таким образом, можно выделить несколько ключевых аспектов, которые будут описаны далее.

Один из самых главных среди них — процесс внедрения методологии DevOps в работу компании. А именно, каким образом происходит достижение этой цели на пути модернизации подхода к работе компании — за счет использования каких программных средств и изменений в культуре рабочего коллектива.

Поскольку ЗАО «Эвалар» — старая компания, история которой началась в период до двухтысячного года, и имеет большое множество старых проектов, использующих в своей основополагающей части инструменты и решения порой столь же несовременные. Подходы к развертыванию и конфигурации инфраструктуры эволюционируют пропорционально медленно. Когда с каждым годом к старой основе добавляются более новые инновационные решения, а конфигурация различных проектов монотонна и в то же время неоднородна, вопросы масштабируемости, надежности и удобства поддержки становятся все более актуальными.

В компании ведется более десятка активных проектов веб-сайтов, нацеленных на предоставление услуг, и каждый из них имеет разный возраст. Разработка одних была начата раньше, а других позже. В отдельных случаях разница может составлять несколько лет. По этой причине технологии и конфигурации, применяемые данными проектами местами сильно разнятся. Решения некоторых проблем, возникавших в ходе разработки и поддержки этих проектов, требовали и внесения изменений в конфигурацию инфраструктуры, на которой они развернуты. Это неизбежно приводит к расхождениям в имеющихся конфигурациях в тех местах, где их не должно быть, а надстройку новых решений произвести труднее.

Все это привело к необходимости кардинального изменения подходов к работе как со стороны разработчиков, так и со стороны администраторов, занимающихся поддержкой инфраструктуры для разрабатываемых проектов.

Члены команд совместно с руководителями провели анализ существующих решений и пришли к выводу, что внедрение методологии DevOps может стать унифицированным подходом к решению описанных выше проблем.

После принятия решения о постепенном переходе к такой модели работы и согласования с вышестоящим руководством, начался процесс модернизации работы компании.

Поскольку для ведения бизнеса важно сохранять стабильность работающих систем, любые вносимые изменения в цикл разработки и конфигурацию инфраструктуры должны затрагивать лишь определенные аспекты работы, и применяться к среде разработки, после подтверждения стабильности в которой, уже применять к производственной среде.

На данный момент в рассматриваемой компании наблюдается низкий уровень внедрения DevOps, обусловленный некоторыми факторами, описанными ранее в данной работе:

- нехваткой навыков (касается методологии и определенных видов инструментов);
- старой архитектурой;
- ограниченной автоматизацией.

Инфраструктура компании выполнена в виде ряда виртуальных машин, развернутых на выделенных серверах в виртуальном ЦОД* на базе дата-центра компании РЕКОНН, расположенного на территории Российской Федерации, в городе Москва.

Веб-страница с описанием услуги виртуального ЦОД выглядит следующим образом:

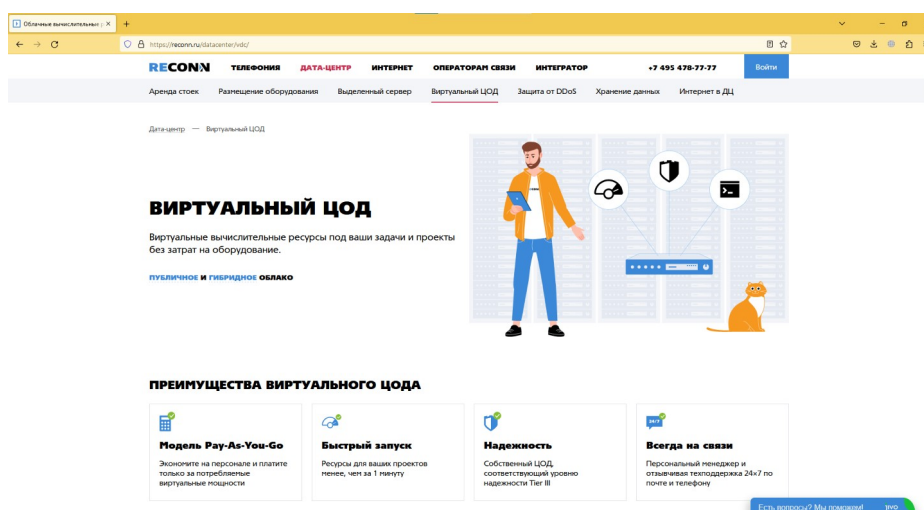


Рисунок 3 — Страница услуги «Виртуальный ЦОД» на сайте РЕКОНН

*Центр обработки данных.

Виртуальный ЦОД позволяет арендовать необходимое количество вычислительных ресурсов — оперативной памяти, ядер процессора, и так далее. Поскольку при таком подходе администратор компании самостоятельно осуществляет развертывание виртуальных машин с необходимыми ресурсами, это позволяет создать гибкую инфраструктуру по наиболее подходящей к нуждам компании модели.[11]

Как и было сказано ранее, серверы проектов компании выполнены на базе виртуальных машин. Для каждого проекта имеется по две виртуальные машины, одна из которых используется в качестве среды разработки, а другая для производственной среды.

Рабочая нагрузка производственной среды развернута без применения технологий контейнеризации, и автоматизация ее конфигурации минимальна. Среда, используемая разработчиками же, напротив, изолирована в контейнеры. Каждый разработчик имеет доступ к своему контейнеру, и к контейнерам других разработчиков для ситуаций, когда требуется оказать помощь коллегам. Эти среды называются стендами разработки. Поскольку могут возникать ситуации, когда отдельный компонент внутри стенда окончательно выходит из строя, либо необходимо развернуть стенд для нового члена команды, процесс развертывания автоматизирован при помощи CI/CD системы Jenkins. По нажатию кнопки запускается CD «пайплайн»[12], развертывающий и подготавливающий новый стенд разработки. Процесс его развертывания включает в себя такие этапы, как подготовка базы данных веб-сайта при помощи специальных сценариев командной оболочки.

Таким образом, суммируя описанное выше текущее состояние инфраструктуры и методов работы с ней, можно выделить несколько серьезных проблем:

- 1) Ручное и частично автоматизированное развертывание и конфигурация. Виртуальные машины, применяемые для стендов разработки и производственных сред разворачиваются вручную, а их конфигурация

осуществляется в полуавтоматическом режиме с применением сценариев командной оболочки. Быстрое создание нескольких виртуальных машин с похожей конфигурацией осуществляется путем развертывания одной виртуальной машины с ее последующим клонированием;

2) Низкая портативность. В случае необходимости развертывания идентичной инфраструктуры на базе другого поставщика услуг или собственном сервере, миграция может занять много времени, будет требовать большого количества ручного вмешательства, а также может стать причиной простоя в работе веб-сайтов;

3) Излишне повышенная нагрузка обслуживающего персонала. По причине наличия большого количества проектов, обслуживаемых гораздо меньшим количеством администраторов, нагрузка на рабочий коллектив возрастает, что также может привести к неблагоприятным последствиям как для обслуживающего персонала, так и для компании в целом;

4) Затрудненность реализации масштабируемости. Такую инфраструктуру невозможно оперативно масштабировать в зависимости от нагрузки и прочих факторов.

5) Отсутствие автоматического поддержания здоровья рабочей нагрузки. При выходе из строя каких-либо сервисов или компонентов, администраторы вынуждены вручную устранять все возникающие типы проблем и повторно приводить сервисы в работоспособное состояние.

2.2 Поиск способов решения выделенных проблем. Разработка плана модернизации

После выделения проблем, которые необходимо решить, можно приступить к созданию плана по модернизации инфраструктуры компании. Необходимо найти подходящие инструменты и методы работы, а также реализовать тестовый вариант модернизированной инфраструктуры для

подтверждения предположений, и дальнейшей проработки проекта, чтобы привести его к следующему этапу.

Для начала следует рассмотреть возможные типы инструментов для решения каждой из задач, после чего сопоставить отдельные инструменты и выбрать среди них наиболее подходящие компании по определенным критериям.

Ручное и частично автоматизированное развертывание и конфигурацию инфраструктуры можно заменить автоматизированными системами, такими как Terraform и Pulumi для развертывания, и Ansible, Chef и Puppet для конфигурации.

Низкая портативность, излишняя нагрузка на рабочий персонал, плохая масштабируемость и отсутствие автоматического поддержания состояния здоровья рабочей нагрузки — все эти проблемы могут быть решены применением систем оркестрации контейнеров, такими как Kubernetes, Nomad и Docker Swarm. Этот набор проблем наиболее приоритетен для решения, их необходимо устранить или минимизировать в первую очередь.

Итак, для выбора одной из систем оркестрации контейнеров, составим сравнительную таблицу. Сравнение будет производиться с применением трех значений веса для определения наличия необходимого функционала: 0 — минимальный или отсутствует, 5 — имеется или достаточно для многих случаев, 10 — имеются широкие возможности. Некоторые ячейки могут содержать вопросительный знак, обозначающий отсутствие достаточного количества информации.

Таблица 1 — Сравнение популярных систем оркестрации контейнеризованных приложений

	Docker Swarm mode	Nomad	Kubernetes
Высокая портативность инфраструктуры с возможностью относительно несложного переноса на другого поставщика или в облако	10	10	10
Открытие дополнительных возможностей автоматизации	5	5	10
Возвращение к предыдущим версиям рабочей нагрузки при возникновении такой необходимости	10	10	10
Широкие возможности по масштабированию рабочей нагрузки и инфраструктуры в целом. В том числе автоматизированно.	5	10	10
Большая устойчивость к проблемам рабочей нагрузки и инфраструктуры за счет самовосстановления	?	10	10
Гибкость выбора отдельных компонентов системы с возможностью их последующей замены	0	0	10
Широкий выбор управляемых облачных решений среди отечественных поставщиков услуг	0	0	10
Гибкие возможности разграничения доступа по ролям	0	10	10
Открытость системы	10	10	10
Широкая распространенность системы	5	5	10

Продолжение Таблицы 1

Исчерпывающая документация	5	10	10
Наличие большого количества обучающих материалов	0	5	10
Zero-downtime развёртывание новых версий и возвращение к предыдущим версиям	10	10	10
Относительная простота архитектуры и скорости проектирования и развёртывания кластеров с нуля	10	10	0

Итого, рассматриваемые системы оркестрации получают следующие баллы:

- 1) Kubernetes — 130;
- 2) Nomad — 105;
- 3) Docker Swarm mode — 70.

Таким образом, наиболее подходящей является система Kubernetes. Поскольку существует множество дистрибутивов этой системы, разрабатываемых разными компаниями и сообществами на базе дистрибутива Kubernetes от Google, называемого «ванильным»* (далее стандартный), далее необходимо выбрать дистрибутив.

Существует организация Cloud Native Computing Foundation (далее CNCF), одним из множества видов деятельности которой является сертификация дистрибутивов Kubernetes. Для внедрения Kubernetes с целью развёртывания критических компонентов инфраструктуры, стоит использовать только сертифицированные дистрибутивы, так как они имеют все необходимые

*«ванильное» программное обеспечение — не подверженное сторонним модификациям, находящееся в своей изначальной форме. Данный термин широко распространен в компьютерной сфере и пришел из английского языка.[13]

базовые компоненты, которые могут понадобиться для работы, что позволит в том числе менять дистрибутивы при условии отсутствия зависимости от дополнительного функционала конкретного поставщика. Также, такие проекты активно разрабатываются и поддерживаются, что позволит избежать использования проекта, который может перестать существовать через какое-то время после введения в использование.

Существует отдельная страница[14] сайта CNCF, на которой любой желающий может посмотреть список сертифицированных вариантов Kubernetes. Среди перечисленных решений существуют как облачные управляемые варианты, так и различные установщики для развертывания на собственной инфраструктуре.

Выбор будет осуществлен на основе сопоставления в Таблице 2 среди трех дистрибутивов Kubernetes, предварительно отобранных членами команды. Система оценивания аналогична таковой в Таблице 1.

Таблица 2 — Выбор дистрибутива Kubernetes

	Стандартный Kubernetes от Google	k3s	RKE
Легкость развертывания	0	10	5
Количество сред выполнения контейнеров на выбор	10	5	0
Возможности модификации относительно модульности компонентов	10	0	5
Простота поддержки и администрирования кластера	0	5	5
Качество и доступность документации	10	5	5
Низкая вероятность привязки к поставщику	10	10	10

Продолжение Таблицы 2

Популярность согласно субъективным наблюдениям	10	5	0
Низкая или нулевая вероятность отклонения от «ванильного» Kubernetes	10	0	?
Сертифицирован CNCF	10	10	10
Отсутствуют субъективные разработки/предустановки/реализация чего-либо	10	0	0
Низкое потребление ресурсов	0	5	0
Небольшое количество действий/компонентов для минимального рабочего развертывания руками	0	5	5

Итого, рассматриваемые дистрибутивы Kubernetes имеют следующие баллы:

- 1) Стандартный Kubernetes — 80;
- 2) k3s — 60;
- 3) RKE — 45.

Таким образом, наиболее подходящим вариантом является стандартный дистрибутив Kubernetes, разрабатываемый компанией Google.

Далее необходимо выбрать, каким методом будет выполняться развертывание и администрирование кластера, так как для выбранного дистрибутива на момент создания данной работы существует три инструмента развертывания и администрирования:

- kubeadm;
- kOps;
- Kubespray.

Эти инструменты позволяют создавать кластеры, готовые для применения в производственной среде.

Инструмент kOps не поддерживает развертывание кластеров на собственной инфраструктуре и нацелен специфически на использование с поставщиками облачных вычислений, поэтому не подходит.

Kubeadm — это инструмент командной строки, применяемый для установки базовых компонентов кластера на серверы, создания, управления и удаления кластера.

Kubespray — инструмент для работы с кластером, используя Ansible.

Поскольку имеется необходимость перехода на модель IaC, инструмент, который должен применяться для работы с кластером, должен конфигурироваться, используя набор файлов с кодом, описывающим желаемое состояние кластера. Среди двух подходящих инструментов таковым является Kubespray. Дополнительным преимуществом является то, что для использования данного инструмента весь необходимый код клонируется из git репозитория. Это означает, что его можно добавить как подмодуль в другой проект по работе с инфраструктурой, использующий Ansible. Это позволит осуществлять все действия из одного места, а также удобно вести разработку и отслеживать историю изменения всего проекта.

Следующее, что нужно выбрать — инструмент для управления конфигурацией инфраструктуры, который сможет обеспечить применение подхода инфраструктуры как кода. Так как модернизацией инфраструктуры будет заниматься небольшая команда, сильно нагруженная задачами по обеспечению работы сервисов компании, и планируется внедрение сложной системы автоматизации, развертывания, масштабирования и управления контейнеризованными приложениями, инструмент управления конфигурацией должен быть более легким в изучении, и обеспечивать быстрое начало работы с ним. Поскольку для развертывания и администрирования кластера выбран Kubespray, основанный на базе Ansible, наиболее подходящий вариант —

Ansible. Это позволит избежать изучения очередного инструмента, и упростить работу с инфраструктурой, так как она будет выполнена, применяя один инструмент.

Далее потребуется выбор инструмента для автоматизированного развертывания инфраструктуры. Этот инструмент должен обладать исчерпывающей документацией, иметь большое количество поставщиков для развертывания инфраструктуры, а также быть гибким, относительно легким в изучении. Выбор будет осуществлен, используя Таблицу 3. Система оценивания аналогична таковой в Таблице 1.

Таблица 3 — Выбор системы автоматизированного развертывания инфраструктуры

	Terraform	Pulumi
Возможность использование известных администраторам языков программирования	0	10
Наличие большого количества поддерживаемых поставщиков инфраструктуры	10	10
Гибкость описания инфраструктуры	5	10
Возможность осуществлять тестирование конфигурации на ошибки	5	10
Возможность использования с распространенными редакторами кода и интегрированными средами разработки	5	10

Продолжение Таблицы 3

Гибкость описания инфраструктуры, используя специальный язык, возможность реализации сложной логики	5	10
--	---	----

Итого, представленные инструменты получили следующие баллы:

- 1) Pulumi — 60;
- 2) Terraform — 30.

Таким образом, наиболее подходящим инструментом является Pulumi.

Имея список технологий и конкретных реализаций, можно приступить к созданию плана модернизации инфраструктуры компании и создания культуры DevOps среди ее сотрудников.

Для более удобного восприятия план лучше представить в виде пронумерованного списка, так как этапы модернизации должны выполняться последовательно.

- 1) Выбрать проект для тренировки. Перед осуществлением переноса всех проектов стоит выбрать самый простой, состоящий из наименьшего числа компонентов. Это упростит задачу, так как позволит сфокусироваться на углублении понимания принципов работы с Kubernetes вместо траты большого количества времени на попытки запуска отдельных компонентов проекта;
- 2) Спроектировать кластер для окружения разработки, его архитектуру. На этом этапе необходимо понять, из какого количества серверов должен состоять кластер, какие ресурсы он должен иметь, а также, какие дополнительные компоненты понадобятся для правильной реализации работы проекта, который будет перенесен в первую очередь;
- 3) Развертывание кластера. После проработки архитектуры, приступить непосредственно к реализации развертывания кластера Kubernetes;

- 4) Осуществить портирование выбранного проекта в Kubernetes. Для этого понадобится выполнить модификации некоторых конфигураций проекта, а также создать новые, необходимые для создания ресурсов API Kubernetes;
- 5) Выполнить перенос стендов разработки. Основываясь на результате портирования проекта, выполнить развертывание стендов разработки;
- 6) Произвести наблюдение за состоянием проекта, периодически собирать информацию от команды разработки для анализа;
- 7) Устранить недочеты, обнаруженные в ходе этапа №6;
- 8) Осуществить этапы №1, №4—6 для оставшихся проектов;
- 9) Внедрить использование инструмента для реализации автоматического развертывания инфраструктуры;
- 10) Продолжительное время, около трех месяцев осуществлять наблюдение за проектом и устранять возможные недочеты;
- 11) Запросить разрешение у вышестоящего руководства на развертывание производственных сред в кластере Kubernetes;
- 12) Осуществить поэтапно для каждого проекта компании этапы аналогичные этапам №1, №5—7 для развертывания производственных сред.

Итого, имеется план, состоящий из двенадцати этапов, которые помогут модернизировать инфраструктуру компании ЗАО «Эвалар».

2.3 Начало реализации плана по модернизации инфраструктуры

2.3.1 Выбор проекта

Как было сказано ранее, проект, с которого планируется начать модернизацию инфраструктуры путем внедрения Kubernetes, должен быть относительно несложным и состоять из наименьшего количества компонентов. В данном случае таким проектом оказался веб-сайт <https://corp.evalar.ru/>, который является визитной карточкой компании. Внешний вид стартовой страницы данного сайта показан на Рисунке 4.

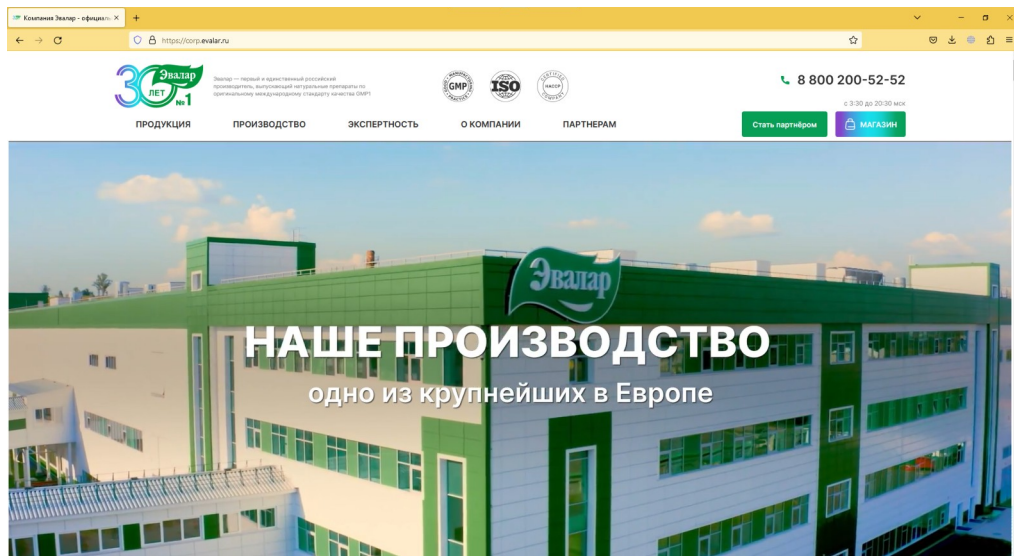


Рисунок 4 — Стартовая страница сайта corp.evalar.ru

Данный проект состоит из трех компонентов:

- Обратный прокси-сервер на базе NGINX;
- Веб-сайт на базе PHP;
- СУБД Persona.

Это означает, что для развертывания в Kubernetes понадобится создать три манифеста для сборки образов контейнеров каждого из этих трех компонентов.

2.3.2 Проектирование кластера

Поскольку после завершения модернизации инфраструктуры с полным переносом всех проектов компании в Kubernetes, еще на этапе проектирования кластера необходимо задуматься о дальнейших возможностях масштабирования и повышении отказоустойчивости.

Так, согласно официальной документации Kubernetes, одним из ключевых факторов достижения высокой доступности кластера Kubernetes является использование внешнего балансировщика сетевого трафика для доступа к API серверу хостов панели от рабочих хостов. Также стоит использовать доменное имя вместо IP-адреса для использования round-robin DNS в случае применения нескольких балансировщиков.

Для хранения данных нужно организовать специально отведенное хранилище. На начальных этапах будет применяться NFS хранилище, разворачиваемое на отдельном сервере, не являющемся участником кластера. В дальнейшем можно будет рассмотреть различные решения специализированных поставщиков хранилища для Kubernetes, таких как Longhorn и Rook. Этот же сервер будет содержать реестр образов контейнеров, который будет использоваться Kubernetes для загрузки приватных образов проектов компании.

По умолчанию сервисы, разворачиваемые в Kubernetes, не могут быть использованы из внешней сети. Для обеспечения такого функционала обычно применяется ресурс типа LoadBalancer. В стандартной реализации отсутствует внешний балансировщик для публикуемых сервисов. Существуют лишь сторонние решения. У каждого поставщика облачных услуг имеется свое внутреннее решение, абстрагированное от пользователей. Для кластеров же, развернутых на собственной инфраструктуре существует не так много продуктов. Один из них MetalLB. Он является самым подходящим, так как не требует большого количества конфигурации и может быть легко установлен, используя Helm*.

Для MetalLB необходимо выделить один свободный IP-адрес в той же сети, в которой находятся участники кластера Kubernetes.

MetalLB состоит из двух компонентов, разворачиваемых в виде подов внутри кластера: контроллер и спикер. Контроллер разворачивается на любом участнике кластера и выполняет функцию по выдаче IP-адресов из пула, заданного администратором, сервисам типа LoadBalancer. Спикер разворачивается на всех участниках кластера и осуществляет анонсирование IP-адреса определенного сервиса, если этот сервис развернут на том же участнике

*Helm — пакетный менеджер для Kubernetes, позволяющий устанавливать «чарты», запакованные манифесты ресурсов Kubernetes, шаблонируемые при помощи специального синтаксиса.[15]

кластера, что спикер. С точки зрения сети это выглядит так, будто сетевой интерфейс участника кластера имеет несколько интерфейсов.[16]

Существует два режима работы MetalLB: L2, BGP.[17] В зависимости от сконфигурированного режима, спикеры осуществляют анонсирование IP-адресов либо при помощи ARP, либо NDP, или используя сетевой протокол BGP соответственно. При использовании режима L2, весь сетевой трафик для определенного сервиса проходит через определенного участника кластера, после чего может быть перенаправлен на другие хосты. В некоторых случаях это может стать проблемой с достижением потолка возможностей сетевого адаптера хоста, что приведет к снижению производительности. Тем не менее, этот режим очень прост в развертывании, он не требует дополнительной конфигурации помимо создания пула адресов. Второй режим работы гораздо более сложен в настройке и требует вмешательства в конфигурацию сетевых устройств для настройки BGP, что позволит осуществлять балансировку сетевого трафика на уровне сетевых устройств.

По причине использования на виртуальных серверах в ЦОД режим BGP не подходит, так как отсутствует доступ к сетевому оборудованию.

Изначально будет достаточно иметь кластер, состоящий из одного хоста панели управления и двух рабочих хостов, которые будут осуществлять подключение к панели управления через внешний балансировщик, брать образы контейнеров с приложениями из приватного реестра на отдельном сервере, и сохранять данные том же сервере.

На Рисунке 5 показана схема архитектуры кластера, который будет развернут. Схема создана в веб-приложении Excalidraw. Подписи выполнены на английском языке для лаконичности и простоты работы. Для более детального рассмотрения данной схемы обратитесь к Приложению А.

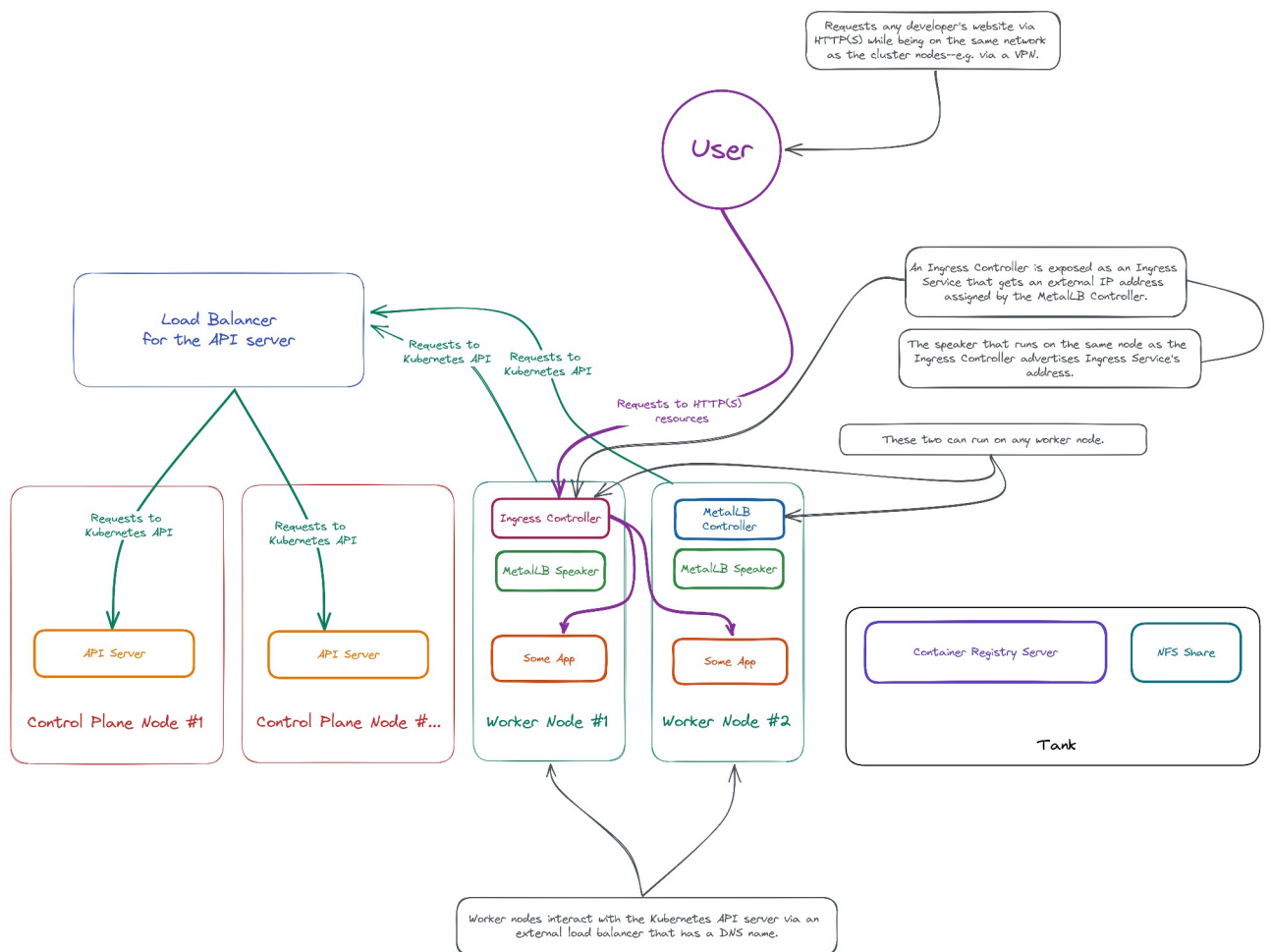


Рисунок 5 — Схема архитектуры кластера

Для подключения разработчиков к стендам разработки, каждому из них потребуется иметь `kubectl` — инструмент для работы с кластером, который будет использоваться для запуска локального прокси, который позволит подключаться внутрь подов с кодом сайтов при помощи SSH через интегрированную среду разработки.

Для ограничения возможностей разработчиков будет применяться RBAC с правилом, разрешающим лишь просматривать поды, запущенные в пространстве имен разработчика, и запускать локальный прокси для определенного сервиса в этом пространстве имен.

2.3.3 Развертывание кластера

Итак, имея необходимую архитектуру кластера, можно приступить непосредственно к реализации его развертывания.

В целях безопасности подробности полных настроек и более точных описаний проекта в целом не могут быть раскрыты, поэтому далее будут приводиться лишь обобщенные описания и отдельные фрагменты кода.

Перед развертыванием кластера необходимо создать виртуальные машины, которые должны соответствовать определенным требованиям по количеству выделенных им ресурсов.

В проектной документации была составлена таблица необходимых требований, согласно которым один из администраторов создал необходимые виртуальные машины с возможностью удаленного доступа через SSH по публичному ключу RSA, который был передан ему. Снимок экрана данной таблице представлен на Рисунке 6.

Хост	Требования	
Балансер	RAM	2 GiB
	CPU Cores	1
	OS	Ubuntu Server 22.04 LTS
Хранилище данных (tank)	RAM	4 GiB
	CPU Cores	4
	OS	Ubuntu Server 22.04 LTS
	Storage	Дополнительный диск для хранилища данных. Нужно много ГБ, ведь под каждого разработчика для каждого проекта уходит несколько ГБ. Начать можно с 32 ГБ и по необходимости расширять.
Хосты панели управления кластера	RAM	4 GiB
	CPU Cores	4
	OS	Ubuntu Server 22.04 LTS
Воркер ноды кластера	RAM	2 GiB
	CPU Cores	2
	OS	Ubuntu Server 22.04 LTS

Рисунок 6 — Системные требования в проектной документации

После развертывания виртуальных машин и выделения дополнительного IP-адреса, который в дальнейшем будет применяться для обращения к Ingress ресурсу кластера для сервисов, к которым необходимо обеспечить доступ извне, согласно проектной документации, администраторы создали записи с необходимыми доменными именами. Снимок экрана с таблицей представлен на Рисунке 7.

1.2.2. Доменные имена

Для чего	На что указывает	Пример паттерна
Внешний балансировщик для Kubernetes API	Адрес балансировщика	<code>balancer-01.devk8s. [REDACTED]</code>
Проект для разработчика. Каждому проекту для каждого разработчика задается свое имя. Все эти имена указывают на один и тот же адрес.	Адрес Ingress-контроллера	<code><фамилия>-<имя>.<код_проекта>.devk8s. [REDACTED]</code> , где <код_проекта> — это сокращенное название проекта. Например, для сайта "[REDACTED]" — "[REDACTED]".

Рисунок 7 — Требования к доменным именам в проектной документации

Проект для работы с инфраструктурой имеет четко установленную структуру, которой должны придерживаться администраторы, которые будут осуществлять работу с данным проектом. Для этого в проектной документации на главной странице существует специальный раздел с описанием этой структуры. Данный материал может быть показан только сотрудникам компании.

Помимо этого в проектной документации имеются отдельные инструкции для разработчиков и быстрый старт для администраторов. Также имеются прочие разделы для более подробного ознакомления с отдельными элементами проекта, не затронутыми в быстром старте.

Поскольку для написания проектной документации используется специализированный язык разметки AsciiDoc, поддерживающий генерацию статических сайтов с документацией, также имеются инструкции по сборке такого статического сайта для проекта. На данный момент каждый

взаимодействующий с проектом сотрудник генерирует документацию локально в HTML файлы, но в дальнейшем планируется настройка GitLab CI для автоматической сборки и публикации в GitLab Pages[18] внутри сети компании.

После получения кода проекта, подготовки хоста управления и проверки доступности подключения к виртуальным серверам, выполняется непосредственно развертывание кластера.

По соображениям безопасности, вывод работы команд не может быть представлен. Поскольку требуется выполнить более десятка команд, представлять их в содержании данной работы нецелесообразно. Для ознакомления с командами по развертыванию кластера Kubernetes, обратитесь к Приложению Б.

2.3.4 Портирование выбранного проекта

После успешного развертывания кластера, можно приступить к портированию выбранного проекта компании, сайта corp.evalar.ru. Для этого необходимо иметь контейнерфайлы с описанием инструкций по сборке образов контейнеров, плейбуки для сборки и выгрузки данных контейнеров в приватный реестр образов контейнеров. Помимо этого необходимо наличие Helm-чарта для развертывания самого проекта, а также плейбуки для автоматизированной установки чарта.

Для проекта созданы контейнерфайлы, представленные на Рисунке 8.

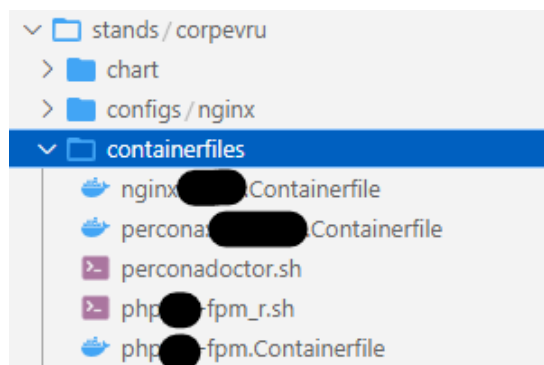


Рисунок 8 — Контейнерфайлы
для портируемого проекта

Как можно заметить, некоторые контейнерфайлы описаны не в формате Dockerfile[19], популяризованном Docker, а как сценарии командной оболочки. Это достигается благодаря использованию инструмента сборки контейнеров Buildah[20]. Такой подход позволяет наиболее гибко выполнять конфигурацию образов, осуществляя получение необходимых дополнительных данных из различных внешних источников, таких как конфигурационные файлы, при этом не выполняя их копирование в создаваемый образ.

На Рисунке 9 показан фрагмент кода одного из таких контейнерфайлов. На нем показан процесс создания пользователей для каждого из разработчиков, получая данные из JSON строки, применяя инструмент jq.

```

85 buildah run --user root $CONTAINER bash -c "$INSTALL_SSH"
86
87 # The DEVELOPERS array parameter expansion is enclosed in double quotes to split array members into words separated by
88 # the first character of the IFS variable – newline (\n) by default.
89 #
90 # https://www.gnu.org/software/bash/manual/html\_node/Arrays.html
91
92 for DEVELOPER in "${DEVELOPERS[@]}"
93 do
94     DEVELOPER_NAME=`jq -cr .name <<< $DEVELOPER`
95     DEVELOPER_PASSWORD_HASH=`jq -cr .passwordHash <<< $DEVELOPER`
96     DEVELOPER_SSH_KEY=`jq -cr .sshKey <<< $DEVELOPER`
97
98     printf 'Setting up developer "%b" ...\n' $DEVELOPER_NAME
99
100     buildah run --user root $CONTAINER useradd $DEVELOPER_NAME -m -s /bin/bash -G $DEVELOPERS_GROUP
101     buildah run --user root $CONTAINER bash -c "chpasswd -e <<< '$DEVELOPER_NAME:$DEVELOPER_PASSWORD_HASH'"
102     buildah run --user root $CONTAINER bash -c "echo $DEVELOPER_SSH_KEY > $SSH_KEYS_DIR/$DEVELOPER_NAME"
103 done
104
105 buildah config --port $SSH_PORT $CONTAINER

```

Рисунок 9 — Фрагмент кода контейнерфайла для генерации аккаунтов пользователей внутри контейнера для последующего создания его образа

Несмотря на относительную простоту и небольшое количество компонентов проекта, портирование которого осуществляется, это потребовало создания большого количества манифестов — девятнадцать файлов, создающих все необходимые ресурсы Kubernetes для успешного развертывания. Суммарный объем данных конфигурационных файлов также оказался большим — пятьсот сорок шесть строк кода (см. Рисунок 10).

```
(~/projects/kubercluster/stands/corpevru/chart)
(16:05:54 on main) → wc -l `find . -type f | grep -vP '(ignore|Chart)'` | tail -1
546 total
```

Рисунок 10 — Суммарное количество строк кода чарта проекта

В качестве примера манифестов Kubernetes на Рисунке ниже приведен манифест для создания роли для разработчиков, которая ограничит их возможности по взаимодействию с API кластера.

```
{...} developer-portforward.yml ×
stands > corpevru > chart > templates > roles > {...} developer-portforward.yml > {map} > apiVersion
1  apiVersion: rbac.authorization.k8s.io/v1
2  kind: Role
3  metadata:
4    name: developer-portforward
5    namespace: {{ .Release.Namespace }}
6
7  rules:
8  - apiGroups: [""]
9    resources: ["services"]
10   resourceNames: ["php-ssh"]
11   verbs: ["port-forward", "get"]
12
13  - apiGroups: [""]
14    resources: ["pods"]
15    verbs: ["list", "get"]
16
17  - apiGroups: [""]
18    resources: ["pods/portforward"]
19    verbs: ["create"]
20
```

Рисунок 11 — Роль для аккаунтов разработчиков

Поскольку в Kubernetes отсутствуют встроенные механизмы управления аккаунтами, необходимо создавать для каждого пользователя сертификаты X.509[21] с определенными полями и подписывать их центром сертификации кластера. Для этого также создана отдельная роль Ansible, позволяющая автоматизировать процесс.

2.3.5 Перенос стендов разработки

После портирования проекта для развертывания в Kubernetes, можно приступить непосредственно к развертыванию стендов разработки для программистов, работающих над данным проектом, чтобы обеспечить им доступ к среде, которая должна заменить их текущую среду разработки.

2.2.1. corpevru

1. Пример: Сборка выбранных образов, и их выгрузка в приватный реестр

```
(kubespray-venv) $ ansible-playbook -i inventories/kubercluster/stage.yml stands/corpevru/images
```

2. Пример: Подготовка файлов проекта для выбранных разработчиков

```
(kubespray-venv) $ ansible-playbook -i inventories/kubercluster/stage.yml stands/corpevru/skel.y
```

3. Развертывание стенда

1. Создание пакета с чартом для сайта

```
(kubespray-venv) $ # В случае успешной сборки будет показано, куда положен пакет. Запомни  
(kubespray-venv) $ cd stands/corpevru && make && cd -
```

2. Пример: Развертывание для всех разработчиков

```
(kubespray-venv) $ # В переменной `package` обязательно нужно указать название файла паке  
(kubespray-venv) $ # Путь к пакету указывается относительно корня подпроекта девстенда со  
(kubespray-venv) $ ansible-playbook -i inventories/kubercluster/stage.yml stands/corpevru,
```



Если вся рабочая нагрузка развернута как надо, и ингресс контроллер получил нужный адрес, но при попытке подключения вылетает ошибка 503, а потом 502, просто немного подождите. Через минуту-другую все обычно начинает работать.

Рисунок 12 — Раздел проектной документации по развертыванию стендов разработки для всех разработчиков

После успешного развертывания в Kubernetes имеется множество новых ресурсов, некоторые примеры из которых показаны на Рисунке 13.

В первом блоке на приведенном снимке экрана отображена информация о ресурсе Ingress. Этот ресурс получает внешний адрес, выделенный ранее, и

используется для осуществления доступа к сервису, развернутому внутри кластера, извне. На данный момент реализована работа только через протокол HTTP, поэтому среди отображаемых портов только порт номер восемьдесят. В дальнейшем планируется реализация поддержки HTTPS с автоматической генерацией сертификатов и их привязкой к нужным ресурсам.

Компоненты развернутого стенда для каждого из разработчиков выглядят примерно так:

Ингресс

```

[redacted]@control-plane-01:~$ kubectl get ingress -n corpevru-timofey-chuchkanov
NAME          CLASS    HOSTS                                     ADDRESS          PORTS
nginx-proxy   nginx    chuchkanov-timofey.corpevru.devk8s.    [redacted]        [redacted]    80
  
```

Рабочая нагрузка, сервисы

```

[redacted]@control-plane-01:~$ kubectl get all -n corpevru-timofey-chuchkanov
NAME          READY   STATUS    RESTARTS   AGE
pod/[redacted]  1/1     Running   0           92m
pod/[redacted]  1/1     Running   0           92m
pod/[redacted]  1/1     Running   0           92m
  
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/[redacted]	ClusterIP	[redacted]	<none>	9000/TCP	92m
service/[redacted]	ClusterIP	[redacted]	<none>	3306/TCP	92m
service/[redacted]	ClusterIP	[redacted]	<none>	80/TCP	92m

Рисунок 13 — Некоторые компоненты развернутой рабочей нагрузки

После успешного развертывания, веб-страницы стендов разработчиков каждого разработчика могут быть просмотрены при обращении по соответствующим доменным именам, а сами разработчики могут осуществить подключение внутрь подов[22] для разработки через SSH после осуществления создания обратного прокси следующей командой*, пример работы которой приведен на Рисунке 14:

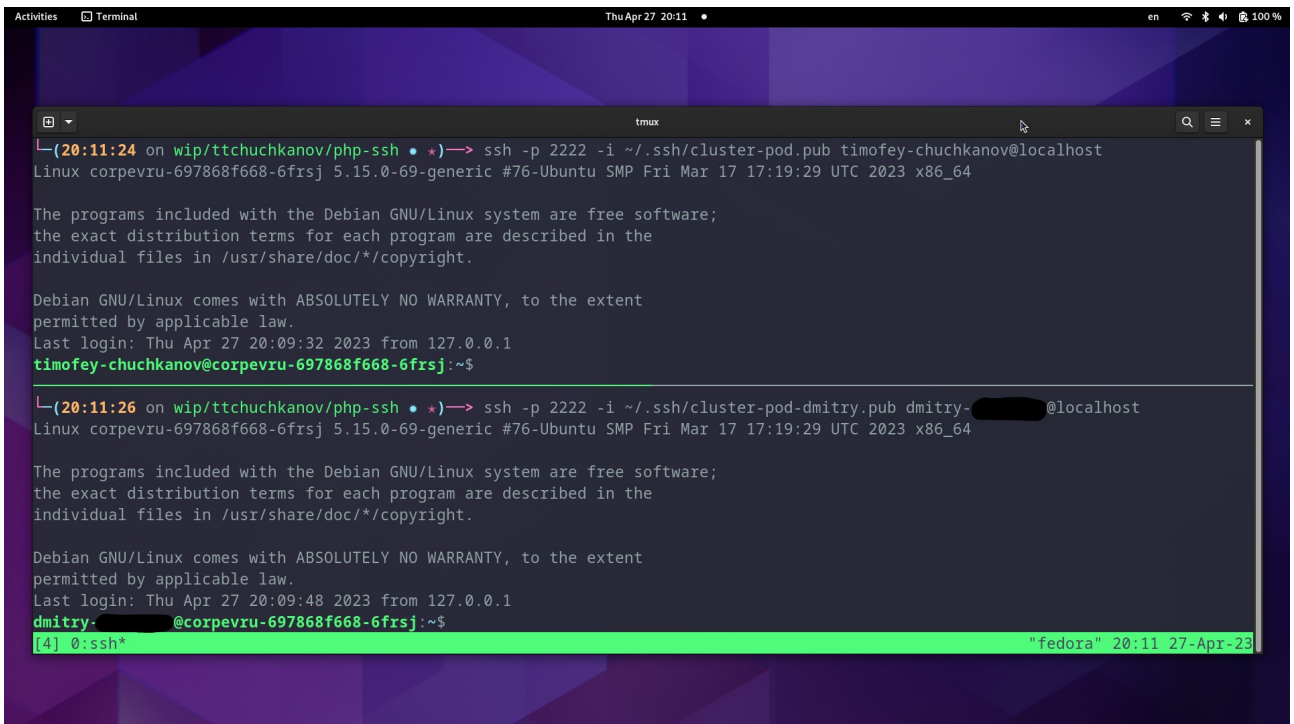
```
$ kubectl port-forward
```

*Текст команды начинается после символа доллара (\$). Такой формат записи означает выполнение команды без повышенных привелегий.

```
(~/projects/Ansible/kubercluster)
(20:08:59 on wip/ttchuchkanov/php-ssh *★)→ kubectl port-forward service/php-ssh 2222:22
Forwarding from 127.0.0.1:2222 -> 22
Forwarding from [::1]:2222 -> 22
Handling connection for 2222
Handling connection for 2222
Handling connection for 2222
Handling connection for 2222
Handling connection for 2222
```

Рисунок 14 — Пример осуществления проброса порта через kubectl

На Рисунке 15 показан пример подключения разработчиков по SSH внутрь одного из подов в кластере.



```
Activities Terminal Thu Apr 27 20:11 • en 100%
tmux
(20:11:24 on wip/ttchuchkanov/php-ssh *★)→ ssh -p 2222 -i ~/.ssh/cluster-pod.pub timofey-chuchkanov@localhost
Linux corpevru-697868f668-6frsj 5.15.0-69-generic #76-Ubuntu SMP Fri Mar 17 17:19:29 UTC 2023 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr 27 20:09:32 2023 from 127.0.0.1
timofey-chuchkanov@corpevru-697868f668-6frsj:~$

(20:11:26 on wip/ttchuchkanov/php-ssh *★)→ ssh -p 2222 -i ~/.ssh/cluster-pod-dmitry.pub dmitry-@localhost
Linux corpevru-697868f668-6frsj 5.15.0-69-generic #76-Ubuntu SMP Fri Mar 17 17:19:29 UTC 2023 x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Apr 27 20:09:48 2023 from 127.0.0.1
dmitry-@corpevru-697868f668-6frsj:~$
[4] 0:ssh* "fedora" 20:11 27-Apr-23
```

Рисунок 15 — Пример подключения к двум аккаунтам разработчиков внутрь одного из подов кластера после осуществления проброса портов

Веб-страница проекта для каждого разработчика выглядит аналогично тому, что представлено на Рисунке 16.

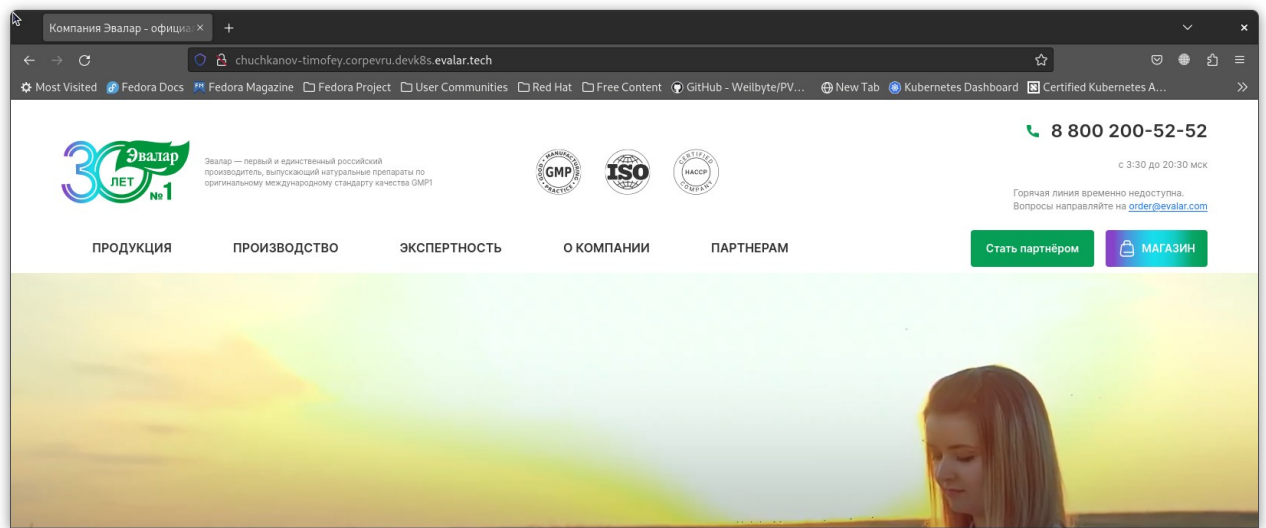


Рисунок 16 — Веб-страница стендов разработки одного из разработчиков, развернутого внутри кластера Kubernetes

2.4 Дальнейшее развитие проекта

Предположения о значительной массивности проекта по модернизации инфраструктуры компании ЗАО «Эвалар» и созданию культуры DevOps оказались подтверждены. Это действительно является серьезным шагом к дальнейшему развитию компании, и требует высокого труда и слаженности работы команд.

На данный момент успешно завершены пять этапов по модернизации:

- 1) Выбор проекта;
- 2) Проектирование кластера;
- 3) Развертывание кластера;
- 4) Портинг выбранного проекта для работы в Kubernetes;
- 5) Перенос стендов разработки в Kubernetes.

В текущем состоянии разработчики проекта веб-сайта corp.evalar.ru могут осуществлять подключение к своим стендам через SSH, используя привычные интегрированные среды разработки, а также любой сотрудник компании, имеющий доступ к сети, в которой расположены виртуальные серверы-участники кластера, имеет возможность осуществить подключение к веб-страницам каждого стенда при помощи протокола HTTP.

Примерные сроки достижения следующих этапов приведены в Таблице 4 в формате ISO 8601-2[23]. В зависимости от возникновения различных ситуаций данные сроки могут быть скорректированы в дальнейшем.

Таблица 4 — Примерные сроки реализации следующих этапов проекта в формате ISO 8601-2

№ Этапа	Сроки
6	2023-06/2023-07
7	2023-07
8	2023-07/2023-09
9	2023-10
10	2023-10/2023-12
11	2024-01
12	2024-01/*

Также, одновременно с реализацией данных этапов, предстоит усилить культуру DevOps среди команд разработки и поддержки инфраструктуры, чтобы добиться максимальной эффективности рабочих процессов.

ГЛАВА 3. ЭКОНОМИЧЕСКАЯ ЧАСТЬ

Поскольку реализация любого проекта по модернизации не должна производиться в ущерб бизнесу, необходимо грамотно рассчитать все возможные затраты.

Так как применяется виртуальный ЦОД от компании РЕКОНН, для подсчета затрат на аренду необходимых вычислительных ресурсов можно воспользоваться удобным калькулятором на сайте. Расчеты приведены на Рисунке 17.

ПУБЛИЧНОЕ ОБЛАКО

Количество vCPU: 14 (1 to 80 шт.)

Память vRAM: 14 (1 to 300 ГБ)

Диск: Base vSSD (7000 IOPS (количество операций ввода-вывода в секунду))

256 (1 to 4 096 ГБ)

в час* **22 Р**

в месяц* **15 840 Р**

ЗАКАЗАТЬ

* Без учета стоимости аренды IP-адресов. Офертой не является.

Рисунок 17 — Расчет затрат на аренду
вычислительных ресурсов

Итого, для изначальной версии проекта, имеющей: один хост панели управления кластера, два рабочих хоста, внешний балансировщик и отдельное хранилище, ежемесячные расходы будут составлять пятнадцать тысяч восемьсот сорок рублей. Данная сумма является удовлетворительной с учетом крупных масштабов компании.

С точки зрения затрат на оплату труда, так как для реализации проекта на работу на бессрочный договор был принят отдельный администратор, ежемесячная сумма будет исходить из оклада этого администратора.

ГЛАВА 4. ОХРАНА ТРУДА И ТЕХНИКА БЕЗОПАСНОСТИ

Для реализации представленного проекта по модернизации инфраструктуры компании ЗАО «Эвалар» предполагается работа с использованием персонального компьютера, расположенного в офисе на закрепленном за сотрудником рабочем месте, либо, в отдельных случаях, на личном или выданном компанией оборудовании удаленно.

Существуют определенные наборы правил и рекомендаций, которые позволяют избежать получения травм и ухудшения здоровья при работе с ПЭВМ*.

Для обеспечения безопасной работы сотрудников на рабочих местах или при выполнении должностных обязанностей удаленно, следует придерживаться определенного набора мер и нормативов.

Существуют требования как для работодателей, так и для сотрудников. В данном случае рассмотрим некоторые из основных требований к сотрудникам.

Согласно Трудовому кодексу Российской Федерации, работник обязан[24]:

- соблюдать правила эксплуатации производственного оборудования;
- осуществлять наблюдение за исправностью используемого оборудования и инструментов;
- соблюдать требования охраны труда;
- уведомлять своего непосредственного руководителя об обнаруженных неисправностях оборудования, используемого сотрудником, в незамедлительном порядке, а также приостановить рабочую деятельность, требующую эксплуатации данного оборудования, до устранения выявленных неполадок.

*ПЭВМ — персональная электронно-вычислительная машина.

ЗАКЛЮЧЕНИЕ

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. Atlassian. What is DevOps? [Электронный ресурс] // Atlassian. URL: <https://www.atlassian.com/devops> (дата обращения: 23.05.2023).
2. Что такое DevOps и зачем он нужен разработчикам [Электронный ресурс]. URL: <https://academy.yandex.ru/journal/что-такое-devops-i-zachem-on-nuzhen-razrabotchikam> (дата обращения: 23.05.2023).
3. Download the 2021 State of DevOps Report | Puppet by Perforce [Электронный ресурс]. URL: <https://www.puppet.com/success/resources/state-of-devops-report> (дата обращения: 23.05.2023).
4. What are virtual machines? | IBM [Электронный ресурс]. URL: <https://www.ibm.com/topics/virtual-machines> (дата обращения: 24.05.2023).
5. What is containerization? [Электронный ресурс]. URL: <https://www.redhat.com/en/topics/cloud-native-apps/what-is-containerization> (дата обращения: 24.05.2023).
6. Container Runtimes Part 1: An Introduction to Container Runtimes - Ian Lewis [Электронный ресурс]. URL: <https://www.ianlewis.org/en/container-runtimes-part-1-introduction-container-r> (дата обращения: 24.05.2023).
7. namespaces(7) - Linux manual page [Электронный ресурс]. URL: <https://man7.org/linux/man-pages/man7/namespaces.7.html> (дата обращения: 25.05.2023).
8. Control Group v2 — The Linux Kernel documentation [Электронный ресурс]. URL: <https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html> (дата обращения: 25.05.2023).
9. What is Configuration Management? | VMware Glossary [Электронный ресурс]. URL: <https://www.vmware.com/topics/glossary/content/configuration-management.html> (дата обращения: 30.05.2023).
10. Atlassian. Configuration management: definition and benefits [Электронный ресурс] // Atlassian. URL:

<https://www.atlassian.com/microservices/microservices-architecture/configuration-management> (дата обращения: 30.05.2023).

11. Что такое виртуальный ЦОД и чем он отличается от виртуального сервера? — документация Виртуальный ЦОД. Руководство пользователя [Электронный ресурс]. URL: https://cloud.ru/ru/docs/vdc/ug/topics/faq/vcd/vdc__vdc-vs.html (дата обращения: 30.05.2023).
12. Основные сведения о CI/CD-пайплайнах | Руководство по CI/CD в TeamCity | JetBrains [Электронный ресурс]. URL: <https://www.jetbrains.com/ru-ru/teamcity/ci-cd-guide/ci-cd-pipeline/> (дата обращения: 30.05.2023).
13. Vanilla software // Wikipedia. 2023.
14. Certified Kubernetes Software Conformance | Cloud Native Computing Foundation [Электронный ресурс]. URL: <https://www.cncf.io/certification/software-conformance/> (дата обращения: 30.05.2023).
15. Helm [Электронный ресурс]. URL: <https://helm.sh/> (дата обращения: 30.05.2023).
16. About MetalLB and the MetalLB Operator - Load balancing with MetalLB | Networking | OpenShift Container Platform 4.9 [Электронный ресурс]. URL: <https://docs.openshift.com/container-platform/4.9/networking/metallb/about-metallb.html> (дата обращения: 29.05.2023).
17. MetalLB, bare metal load-balancer for Kubernetes [Электронный ресурс]. URL: <https://metallb.universe.tf/configuration/> (дата обращения: 30.05.2023).
18. GitLab Pages | GitLab [Электронный ресурс]. URL: <https://docs.gitlab.com/ee/user/project/pages/> (дата обращения: 30.05.2023).
19. Dockerfile reference | Docker Documentation [Электронный ресурс]. URL: <https://docs.docker.com/engine/reference/builder/> (дата обращения: 30.05.2023).

20. buildah.io [Электронный ресурс] // buildah.io. URL: <https://buildah.io> (дата обращения: 30.05.2023).
21. X.509 : Information technology - Open Systems Interconnection - The Directory: Public-key and attribute certificate frameworks [Электронный ресурс]. URL: <https://www.itu.int/rec/T-REC-X.509> (дата обращения: 30.05.2023).
22. Pods [Электронный ресурс] // Kubernetes. URL: <https://kubernetes.io/docs/concepts/workloads/pods/> (дата обращения: 30.05.2023).
23. ISO 8601 // Wikipedia. 2023.
24. ТК РФ Статья 215. Обязанности работника в области охраны труда \ КонсультантПлюс [Электронный ресурс]. URL: https://www.consultant.ru/document/cons_doc_LAW_34683/d9e91b55bfaa1f838e128d214a7457657f9d56a1/ (дата обращения: 30.05.2023).